



LibrarySQLInjection

Progetto per il corso Sicurezza - A.A. 2025/2026

Docente: Emiliano Casalicchio

9 giugno 2026

Romolo Deffereria
deffereria.2114887@studenti.uniroma1.it

Antonio Bisiach
bisiach.2087632@studenti.uniroma1.it

David Jossua Jaramillo Castillo
jaramillocastillo.1859934@studenti.uniroma1.it

Indice

1	Introduzione e Setup dell'Ambiente	2
1.1	Obiettivo del Progetto e Dominio Applicativo	2
1.2	Scelte Architetture e Tecnologiche	2
1.3	Containerizzazione tramite Docker	2
1.4	Progettazione e Popolamento del Database	3
1.5	Implementazione del Codice Vulnerabile	3
2	Analisi ed Esecuzione degli Attacchi: SQL Injection	4
2.1	Fase Preliminare: Raccolta Informazioni	4
2.2	Violazione della Confidenzialità	5
2.2.1	Attacco 1: Tautologia (Violazione degli Accessi)	5
2.2.2	Attacco 2: Union-Based (Esfiltrazione dei Dati)	6
2.3	Violazione di Integrità e Disponibilità	7
2.3.1	Attacco 3: Query PiggyBacked (Distruzione dei Dati)	7
3	Implementazione delle Difese e Prevenzione degli Attacchi	7
3.1	Configurazione del Database e Permessi	8
3.2	Difesa dalle SQL Injection (Prepared Statements)	8
3.3	Blocco delle Query Multiple e Sicurezza delle Sessioni	9
3.4	Gestione degli Errori	9
4	Testing Finale e Validazione delle Difese	10
4.1	Prevenzione dell'Information Leakage: Mascheramento del DBMS	10
4.2	Validazione della Confidenzialità: Attacco 1 (Tautologia)	10
4.3	Validazione della Confidenzialità: Attacco 2 (UNION-Based)	11
4.4	Validazione di Integrità e Disponibilità: Attacco 3 (Piggybacking)	12
4.5	Conclusioni	13

1 Introduzione e Setup dell'Ambiente

1.1 Obiettivo del Progetto e Dominio Applicativo

Il presente elaborato descrive la progettazione, l'implementazione e l'analisi di sicurezza di un'applicazione web per la gestione integrata di una biblioteca, denominata *Biblioteca del Quartiere*. Sebbene da un lato bisogna che si realizzi un sistema funzionante capace di gestire il catalogo, l'inventario fisico e i flussi di prestito e sanzione; dall'altro, l'obiettivo principale del progetto è quello di sviluppare deliberatamente un ambiente vulnerabile per studiare in modo empirico gli attacchi di tipo SQL Injection (SQLi) e dimostrare la compromissione della triade CIA (Confidenzialità, Integrità, Disponibilità) del database.

1.2 Scelte Architetturelle e Tecnologiche

Per la realizzazione della WebApp si è optato per uno stack tecnologico moderno, ma volutamente configurato per non mitigare le minacce di base:

- **Backend (Python e Flask):** Il linguaggio Python (versione 3.10) è stato scelto per la sua versatilità, accoppiato al micro-framework Flask. Flask risulta ideale per questo scopo didattico poiché, essendo minimalista, permette agli sviluppatori di scrivere e gestire manualmente le query SQL interagendo con il database.
- **Database (MySQL 8.0):** Come RDBMS (Relational Database Management System) si è optato per MySQL, uno standard industriale che offre un ricco set di funzionalità facilmente interrogabili (e sfruttabili) tramite SQL.
- **Frontend (HTML, CSS, Jinja2):** L'interfaccia utente è stata curata per fornire un'esperienza realistica (simulando un vero portale bibliotecario), utilizzando il motore di templating Jinja2 per il rendering dinamico dei dati forniti dal backend.

1.3 Containerizzazione tramite Docker

Per garantire la totale riproducibilità dell'ambiente e isolare il sistema operativo host da potenziali danni collaterali durante la fase di *testing*, l'intera infrastruttura è stata containerizzata utilizzando **Docker** e orchestrata tramite **docker-compose**. Al fine di permettere un confronto simultaneo tra la compromissione e l'efficacia delle contromisure, l'architettura è stata sdoppiata in due ambienti paralleli e isolati:

1. **L'Ambiente Vulnerabile** (`db_vulnerable` e `web_vulnerable`): basato sullo script `initAttacco.sql`, mappa i volumi dell'applicazione ed espone il servizio insicuro sulla porta 9000 dell'host.
2. **L'Ambiente Sicuro** (`db_secure` e `web_secure`): inizializzato con i permessi limitati di `init.sql`, espone la medesima applicazione bonificata sulla porta 9001.

1.4 Progettazione e Popolamento del Database

Il database relazionale (`library_db`) è stato progettato per mappare accuratamente l'ecosistema della libreria. Implementa un controllo degli accessi basato sui ruoli (RBAC), gestito tramite la tabella `Utente` e la differenziazione tra `Cliente` e `Bibliotecario`. Le entità principali includono:

- **Libro e Copia:** Per separare l'opera letteraria astratta dal singolo volume fisico identificabile univocamente.
- **Prestito e Sanzione:** Le entità transazionali che legano gli utenti fisici alle copie nel tempo, gestendo scadenze, ritardi e le relative penali economiche. Per permettere i test di intrusione fin dal primo avvio, il database è stato popolato preventivamente con un set di dati fittizi, comprendente credenziali di amministratori (es. *Mario Rossi*) e di utenti standard.

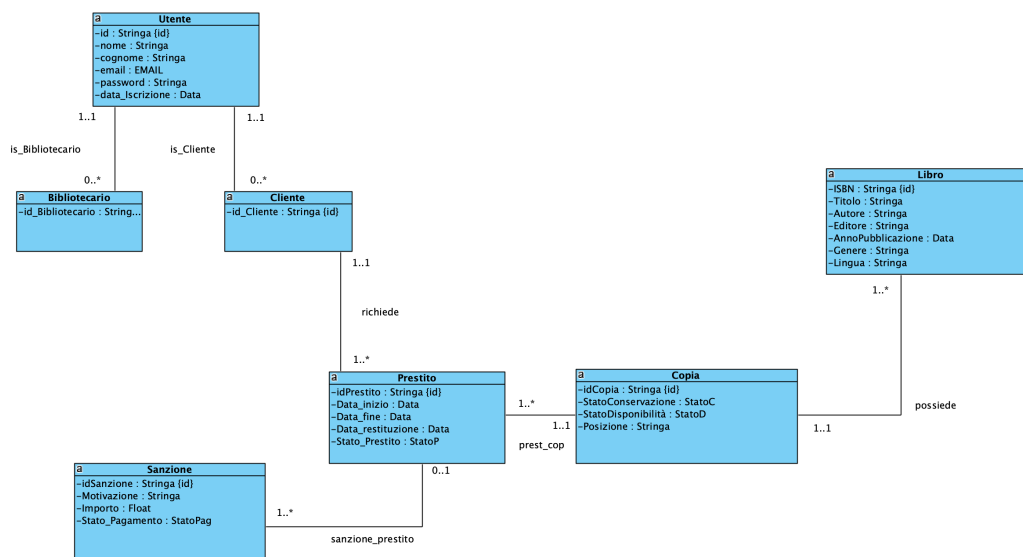


Figura 1: Struttura del Database

1.5 Implementazione del Codice Vulnerabile

Per soddisfare le specifiche della traccia, l'applicazione Flask è stata intenzionalmente progettata con gravi falle di sicurezza nel livello di interazione col database, contravvenendo alle best practice di programmazione sicura. Le due vulnerabilità architetturali inserite nel file `app.py` sono:

1. **Assenza di Parametrizzazione (Uso di f-strings):** Invece di delegare al driver (`mysql-connector`) la sanitizzazione degli input utente tramite *Prepared Statements*, i parametri provenienti dalle richieste HTTP (come le credenziali di login o la barra di ricerca del catalogo) sono stati iniettati direttamente nelle stringhe delle query tramite concatenazione e f-strings (es. `f"SELECT [...] WHERE`

`email='{email}'`). Questo espone il sistema agli attacchi di Tautologia, Commento di fine riga e UNION-based injection.

2. **Abilitazione delle Query Multiple (`multi=True`):** Il driver MySQL per Python, di default, esegue una sola query per volta per motivi di sicurezza. Al fine di permettere attacchi avanzati e distruttivi, come le *Stacked Queries* (o query Piggybacked), si è forzato il metodo `cursor.execute()` abilitando il parametro `multi=True`. Senza questa specifica direttiva, l'interprete Python avrebbe bloccato i tentativi dell'attaccante di concatenare istruzioni DDL o DML (come `UPDATE` o `DROP TABLE`) successive al punto e virgola.

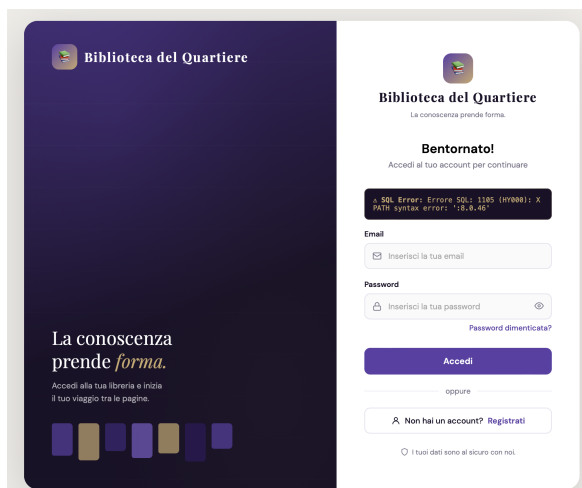
2 Analisi ed Esecuzione degli Attacchi: SQL Injection

In questa fase, il sistema precedentemente configurato è stato sottoposto a un Penetration Test mirato allo sfruttamento delle vulnerabilità del Data Access Layer. L'obiettivo è dimostrare empiricamente come l'assenza di controlli sugli input utente possa portare alla compromissione totale del database, violando i tre pilastri della sicurezza informatica: Confidenzialità, Integrità e Disponibilità (Triade CIA).

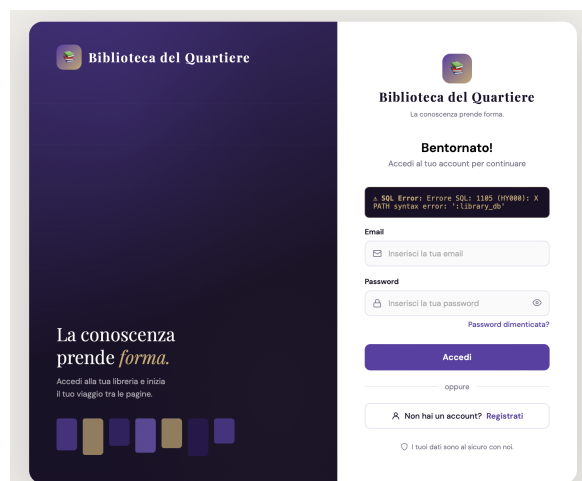
2.1 Fase Preliminare: Raccolta Informazioni

Prima di lanciare un attacco distruttivo, un attaccante necessita di mappare l'infrastruttura bersaglio. A tale scopo, è stata sfruttata una vulnerabilità di tipo *Error-Based SQL Injection* partendo dal modulo di login dell'applicazione. La tecnica consiste nell'inserire caratteri speciali non sanitizzati, come l'apice singolo (`'`), per forzare un errore di sintassi nel motore di database. Poiché l'applicazione è configurata in modo insicuro, essa riflette l'errore generato direttamente nell'interfaccia utente (ad esempio mostrando il messaggio *Errore SQL: 1064 (42000)*), confermando all'attaccante l'utilizzo di MySQL come DBMS sottostante. Una volta confermata la vulnerabilità, l'esecuzione pratica ha previsto l'uso della funzione `EXTRACTVALUE()` con percorsi XPath deliberatamente malformati per estrarre preziose informazioni di sistema e di configurazione. Nello specifico, interrogando il server, sono stati ottenuti:

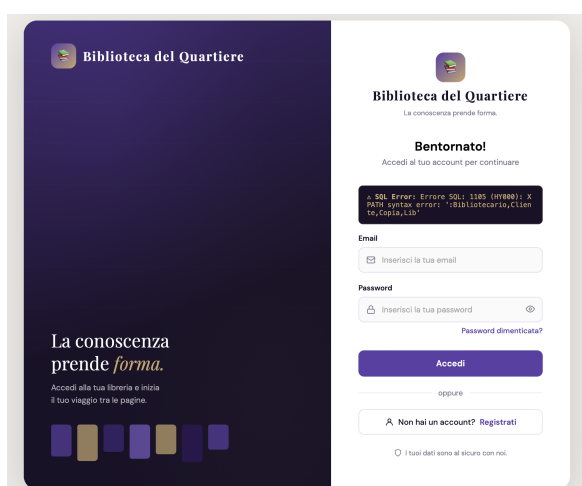
- **La versione esatta del DBMS**, iniettando il payload:
`' AND EXTRACTVALUE(1, CONCAT(0x3a, @@version)) --`
- **Il nome del database in uso**, tramite il comando:
`' AND EXTRACTVALUE(1, CONCAT(0x3a, database())) --`
- **I metadati della struttura relazionale**, interrogando il dizionario interno di MySQL (`information_schema.tables` e `information_schema.columns`). Questo passaggio avanzato ha permesso all'attaccante di individuare a colpo sicuro la tabella bersaglio (*Utente*) e i nomi esatti dei suoi campi sensibili (*nome*, *cognome*, *email*, *password*).



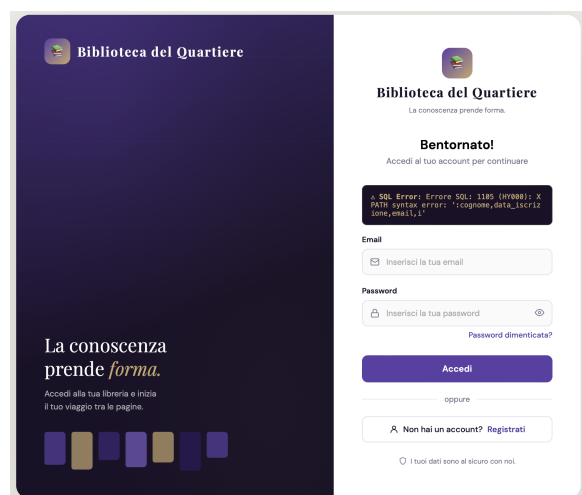
(a) Estrazione versione (8.0.46)



(b) Estrazione nome DB



(c) Estrazione tabelle



(d) Estrazione colonne

Figura 2: Sequenza di esfiltrazione dati tramite *Error-Based SQLi* ed `EXTRACTVALUE()`.

2.2 Violazione della Confidenzialità

2.2.1 Attacco 1: Tautologia (Violazione degli Accessi)

Il primo attacco vero e proprio sfrutta la tecnica dell'Authentication Bypass tramite Tautologia. Questo metodo si basa sulla manipolazione di una query di autenticazione al fine di renderla sempre vera a livello logico, neutralizzando così le clausole che verificano la correttezza delle credenziali inserite. Il successo di questa operazione compromette direttamente il parametro della **Confidenzialità**, in quanto permette a un soggetto esterno di ottenere l'accesso ad aree riservate del sistema senza disporre di un'identità legittima. L'attacco è stato condotto all'interno del form di Login. Inserendo nel campo email il seguente payload:

```
' OR 1=1 --
```

accompagnato da una password arbitraria, la query originale interpolata dal backend è stata alterata in:

```
WHERE u.email='', OR 1=1 -- ' AND u.password='a'
```

Poiché l'espressione `1=1` risulta sempre vera, il controllo dell'email viene superato con successo; contestualmente, l'uso del commento di fine riga (`--`) obbliga il parser di MySQL a ignorare del tutto il successivo blocco di codice relativo al controllo sulla password. In questo modo, l'attaccante è riuscito a effettuare l'accesso impersonando il primo record della tabella, corrispondente all'amministratore del sistema, acquisendone in un solo colpo i privilegi massimi.

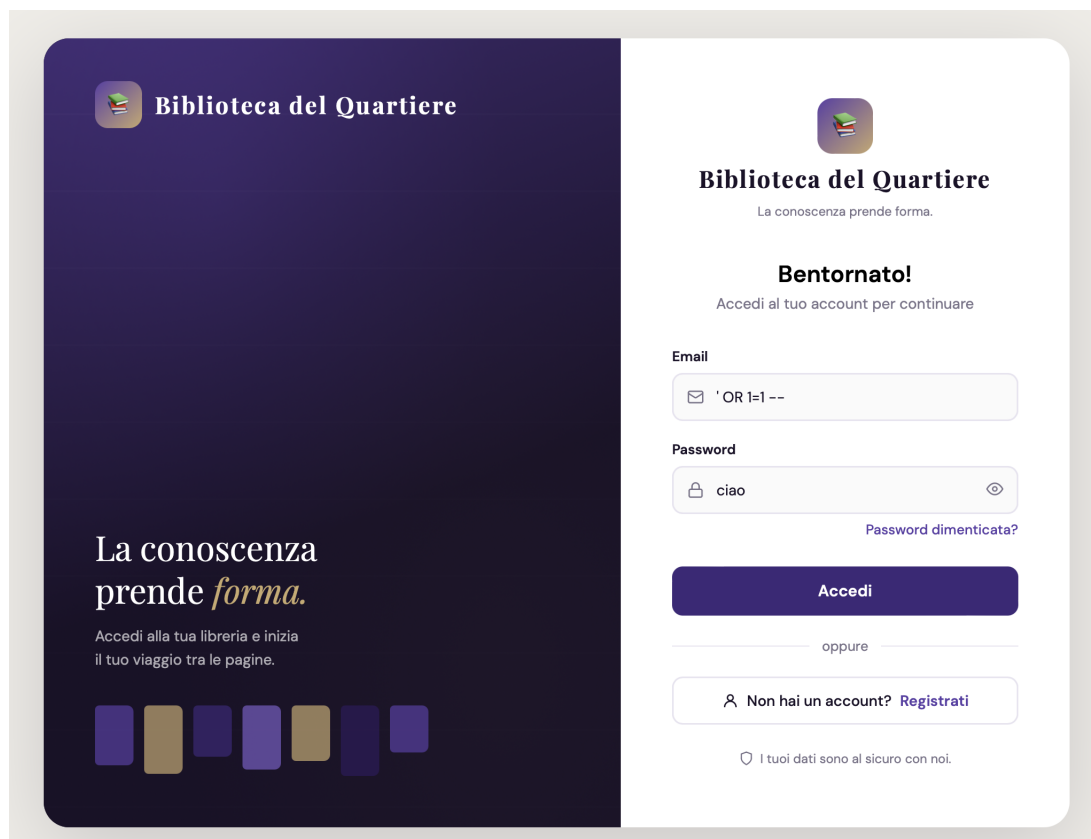


Figura 3: Attacco tramite Tautologia

2.2.2 Attacco 2: Union-Based (Esfiltrazione dei Dati)

Il secondo step ha mirato al furto massivo di informazioni tramite un attacco UNION-Based. Questa sofisticata tecnica si avvale dell'operatore SQL `UNION` per accodare i risultati di una query malevola (composta dall'attaccante) ai risultati generati dalla query legittima dell'applicazione. Il prerequisito architetturale per il suo funzionamento è che entrambe le istruzioni selezionino esattamente il medesimo numero di colonne. Anche in questo caso, la proprietà della Triade CIA ad essere violata è la **Confidenzialità**, poiché dati altamente sensibili, quali password e indirizzi email, vengono esfiltrati dal database e resi leggibili sull'interfaccia pubblica. L'esecuzione ha avuto luogo sfruttando la barra di ricerca del Catalogo, accessibile dal profilo Amministratore. Dopo un'attenta analisi

volta a determinare il numero esatto di colonne restituite dalla query legittima (risultate essere 9 a causa della presenza di funzioni matematiche di aggregazione come COUNT e SUM), è stato iniettato il payload definitivo:

```
x%' GROUP BY 1.ISBN UNION SELECT nome, email, password, 4, 5,
6, 7, 8, 9 FROM Utente -- -
```

Questa iniezione ha terminato prematuramente la ricerca bloccando i risultati originali, ha ripristinato la sintassi corretta tramite la clausola **GROUP BY**, e ha unito in coda l'intero contenuto della tabella **Utente**. Di conseguenza, l'interfaccia grafica web ha elaborato i dati malevoli popolandolo la tabella del catalogo con le email e le password in chiaro degli iscritti, esponendole al posto dei normali metadati dei libri.

2.3 Violazione di Integrità e Disponibilità

2.3.1 Attacco 3: Query PiggyBacked (Distruzione dei Dati)

Il terzo e ultimo attacco ha dimostrato la severa compromissione delle proprietà di **Integrità** e **Disponibilità** del sistema mediante l'uso di *Stacked Queries* (conosciuto anche come Piggybacking). Questa tecnica permette a un utente malintenzionato di eseguire istruzioni SQL multiple e sequenziali all'interno di un'unica richiesta HTTP, incatenandole con un semplice punto e virgola (;). È importante sottolineare che tale vettore di attacco risulta sfruttabile unicamente perché il driver di connessione dell'applicazione (**mysql-connector**) è stato inizializzato con il parametro **multi=True**, abilitando di fatto l'esecuzione di query distruttive a cascata. L'azione distruttiva ha sfruttato la vulnerabilità presente nel form di inserimento "Nuovo Prestito". Per aggirare preventivamente le restrizioni della validazione dei dati lato client (HTML5), l'attaccante si è avvalso degli strumenti di ispezione del browser per alterare la tipologia del campo "Data scadenza", trasformandolo da un rigido calendario (**date**) a una stringa di testo libera (**text**). Ottenuto l'accesso in scrittura, è stato inviato il seguente payload armato:

```
2026-12-31', 'in corso'); DELETE FROM Sanzione; -- -
```

La prima porzione di questo codice ha chiuso in modo pulito e silente l'istruzione di inserimento del prestito (**INSERT INTO**), mentre la seconda ha innescato un comando DML (Data Manipulation Language) che ha cancellato fisicamente tutti i record finanziari dalla tabella **Sanzione**, alterando irreversibilmente l'integrità dei dati. A corollario di questa fase, è stato dimostrato che qualora si fosse impiegato il comando strutturale DDL **DROP TABLE Sanzione;**, l'attacco avrebbe disintegrato l'intera entità logica dal database. In tale scenario, l'applicazione web avrebbe restituito un errore critico (Internal Server Error 500) a chiunque tentasse di accedere alla propria dashboard, generando un Denial of Service (DoS) e sancendo così il collasso totale della **Disponibilità** del servizio.

3 Implementazione delle Difese e Prevenzione degli Attacchi

Dopo aver verificato che gli attacchi descritti nella Fase 2 funzionavano correttamente, siamo passati alla fase di correzione del codice. L'obiettivo di questa terza parte è stato

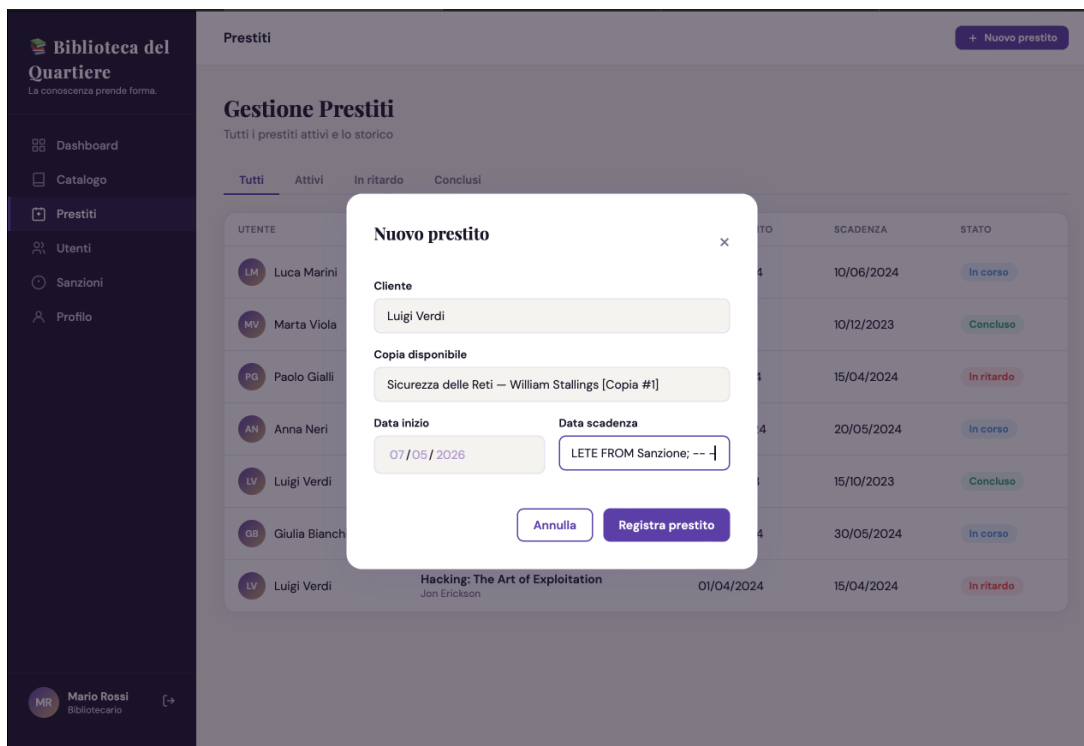


Figura 4: Attacco tramite Piggybacking

quello di mettere in sicurezza l'applicazione, andando a risolvere le vulnerabilità che noi stessi avevamo inserito sia a livello di configurazione del database, sia nel codice Python.

3.1 Configurazione del Database e Permessi

Come prima linea di difesa, abbiamo modificato lo script `init.sql`. Nella versione vulnerabile del progetto, l'applicazione si connetteva al database utilizzando l'utente `root`. Questo è un errore molto comune ma pericoloso, perché in caso di intrusione l'attaccante ha il controllo totale sul DBMS. Per risolvere il problema, abbiamo applicato il **Principio del Minimo Privilegio**.

- Abbiamo creato un utente specifico per l'applicazione chiamato `web_biblioteca`.
- Tramite il comando `GRANT`, gli abbiamo assegnato solo i permessi strettamente necessari per far funzionare il sito: `SELECT`, `INSERT` e `UPDATE`.

In questo modo, abbiamo neutralizzato alla radice l'Attacco 3 (quello distruttivo). Anche se un utente malintenzionato riuscisse a iniettare un comando come `DROP TABLE Sanzione;`, MySQL bloccherebbe l'operazione restituendo un errore di permessi negati, proteggendo così i dati da cancellazioni accidentali o dolose.

3.2 Difesa dalle SQL Injection (Prepared Statements)

Il problema principale nel file `app.py` originale era il modo in cui venivano costruite le query. L'uso delle f-strings di Python (ad esempio `WHERE email='{email}'`) permetteva

all'input dell'utente di mescolarsi con il codice SQL, rendendo possibili gli attacchi visti in precedenza. Per mettere in sicurezza le funzioni di accesso al database (come il login, la ricerca nel catalogo e l'inserimento dei prestiti), abbiamo riscritto le query utilizzando i **Prepared Statements**:

- Abbiamo sostituito le variabili inserite direttamente nelle stringhe con i segnaposti parametrici (%s).
- I dati inseriti dall'utente vengono ora passati al driver MySQL come una tupla separata dalla struttura della query.

Grazie a questa modifica, il database tratta l'input dell'utente esclusivamente come semplice testo e non più come codice eseguibile. Un payload come `' OR 1=1 --` non "romperà" più la query, ma verrà cercato letteralmente nel database (facendo ovviamente fallire la ricerca), bloccando di fatto gli attacchi di Tautologia e le UNION-Based SQLi.

3.3 Blocco delle Query Multiple e Sicurezza delle Sessioni

Per quanto riguarda la logica applicativa, abbiamo sistemato due configurazioni che esponevano il sistema a rischi:

- **Rimozione di `multi=True`:** Nei comandi di esecuzione del cursore avevamo inserito questo parametro per permettere l'esecuzione di più comandi SQL separati dal punto e virgola (;), rendendo possibile l'attacco di *Piggybacking*. Rimuovendolo da tutto il file `app.py`, il driver di connessione torna al suo comportamento standard, accettando una sola query per volta e bloccando i tentativi di concatenare comandi imprevisti.
- **Chiave segreta di Flask:** Nel codice vulnerabile utilizzavamo una `secret_key` statica e banale. L'abbiamo sostituita con una chiave sicura generata dinamicamente ad ogni avvio tramite `os.urandom(24)`. Questo impedisce a un attaccante di indovinare la chiave e falsificare i cookie di sessione per cambiare il proprio ruolo da cliente a bibliotecario.

3.4 Gestione degli Errori

Infine, abbiamo corretto la gestione delle eccezioni nei blocchi `try-except`. Nella prima versione, quando il database andava in errore (ad esempio per una sintassi SQL sbagliata), l'applicazione stampava l'errore tecnico a schermo. Questo era esattamente ciò che ci ha permesso, nella fase di ricognizione, di scoprire la struttura interna del database. Nella versione definitiva, gli errori SQL originali vengono intercettati e stampati solo sulla console del server per comodità di noi sviluppatori (`print(f"Errore SQL: {ex}")`). All'utente finale, invece, viene restituito un messaggio di errore generico (come "Si è verificato un errore"). Nascondendo questi dettagli, si impedisce a un potenziale attaccante di esplorare il database alla cieca.

4 Testing Finale e Validazione delle Difese

A conclusione del ciclo di sviluppo e del successivo processo di irrobustimento dell'applicativo, è stata condotta una sessione di verifica finale. L'obiettivo è certificare l'effettiva risoluzione delle vulnerabilità documentate, dimostrando che le contromisure adottate garantiscono la totale preservazione della Triade CIA. La validazione è stata condotta analizzando la risposta del sistema su due fronti: la prospettiva dell'attaccante (interfaccia utente) e la prospettiva dell'amministratore (log di backend).

4.1 Prevenzione dell'Information Leakage: Mascheramento del DBMS

Nella fase di ricognizione iniziale, l'attaccante era in grado di mappare la struttura del database forzando la generazione di errori SQL a schermo, una vulnerabilità classificata come *Information Exposure Through an Error Message* (CWE-209). Il primo test ha verificato l'efficacia della nuova policy di gestione delle eccezioni. Replicando l'inserimento di caratteri speciali non sanitizzati, l'applicazione non espone più lo stack trace del database. Il blocco `try/except` intercetta l'eccezione nativa e la devia esclusivamente sulla console del server, disaccoppiando l'errore tecnico dal messaggio visualizzato sull'interfaccia:

```
except Exception as ex:
    # L'errore reale viene loggato solo nel backend
    print(f"Dettaglio Errore (Log): {str(ex)}")

    # All'attaccante viene mostrato un rifiuto standard
    error = "Si e' verificato un errore durante l'
autenticazione"
    return render_template('login.html', error=error)
```

Questa configurazione rende il sistema opaco, impedendo all'attaccante di dedurre l'utilizzo di MySQL o di estrarre metadati tramite la funzione `EXTRACTVALUE()`.

4.2 Validazione della Confidenzialità: Attacco 1 (Tautologia)

Il test sulla Tautologia ha replicato l'iniezione del payload `' OR 1=1 --` a all'interno del modulo di login, che in precedenza permetteva l'accesso privilegiato. La difesa è garantita dall'implementazione dei *Prepared Statements*. Rimuovendo le *f-strings*, i parametri utente vengono passati al driver MySQL tramite segnaposti `%s`. Questo approccio impone una rigida separazione tra il codice (la query) e i dati forniti dall'utente:

```
query = """ SELECT u.*, b.id as bib_id, c.id as cli_id
FROM Utente u
LEFT JOIN Bibliotecario b ON u.id=b.id_utente
LEFT JOIN Cliente c ON u.id=c.id_utente
WHERE u.email=%s AND u.password=%s """

cursor.execute(query, (email, password))
```

Il motore del database analizza e definisce la struttura logica della query prima ancora di leggere i parametri. Di conseguenza, il DBMS elabora il payload non come nodo logico, ma come una pura stringa letterale. Cercando nel database un'email testualmente uguale a ' OR 1=1 -- a, la query non trova corrispondenze e l'applicazione nega l'accesso.

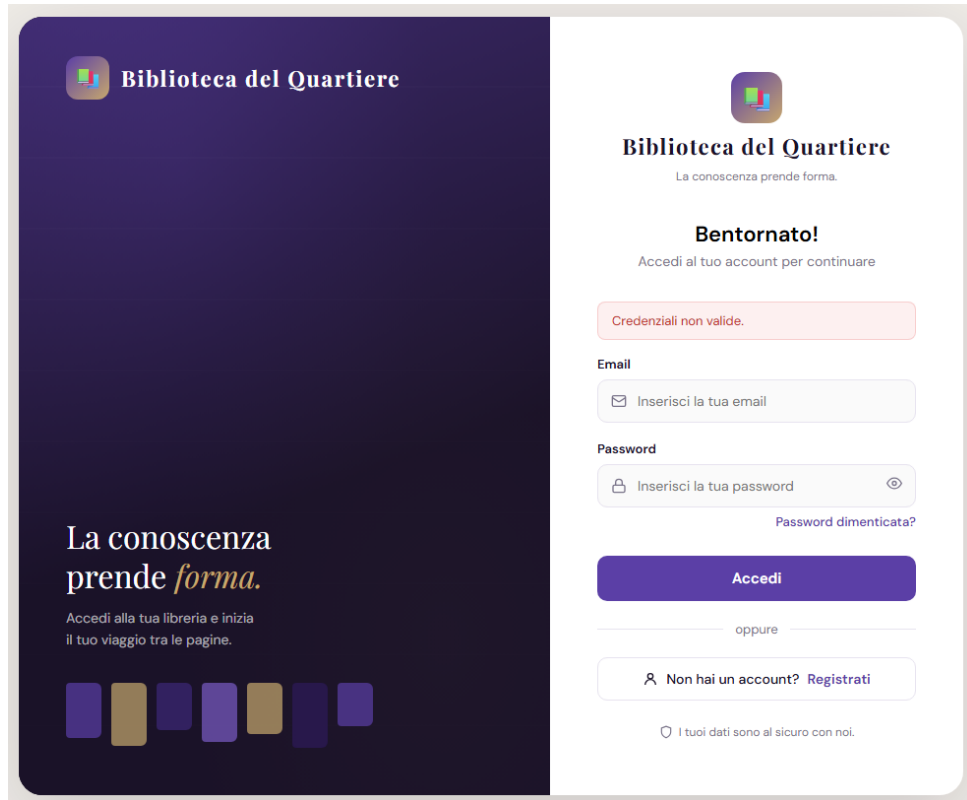


Figura 5: Fallimento dell'attacco di Tautologia: il sistema restituisce il messaggio generico Credenziali non valide.

4.3 Validazione della Confidenzialità: Attacco 2 (UNION-Based)

Il secondo test ha tentato di replicare l'esfiltrazione dei dati sensibili degli utenti (password ed email) dalla barra di ricerca del catalogo librario, precedentemente vulnerabile alle iniezioni di tipo UNION. Anche in questo scenario, l'adozione dei *Prepared Statements* neutralizza la minaccia alla radice. I valori di ricerca vengono preventivamente formattati (con i caratteri jolly %) e iniettati in modo sicuro nella query del Catalogo:

```
query = """ SELECT l.*, COUNT(co.id) as n_copie, SUM(CASE
                WHEN co.statoDisponibilita='disponibile' THEN 1
            ELSE 0 END) as copie_disp
            FROM Libro l LEFT JOIN Copia co ON l.ISBN=co.ISBN
            WHERE l.titolo LIKE %s OR l.autore LIKE %s OR l.
ISBN LIKE %s
            GROUP BY l.ISBN """

val_ricerca = f"%{ricerca}%"
```

```
params = (val_ricerca, val_ricerca, val_ricerca)
cursor.execute(query, params)
```

Il costrutto LIKE %s priva comandi come UNION SELECT del loro significato semantico, costringendo il database a valutarli alla stregua di semplici parole chiave testuali all'interno dei campi Titolo o Autore. L'operazione risulta sintatticamente innocua e restituisce un insieme vuoto. La Confidenzialità dei record della tabella *Utente* è totalmente preservata.

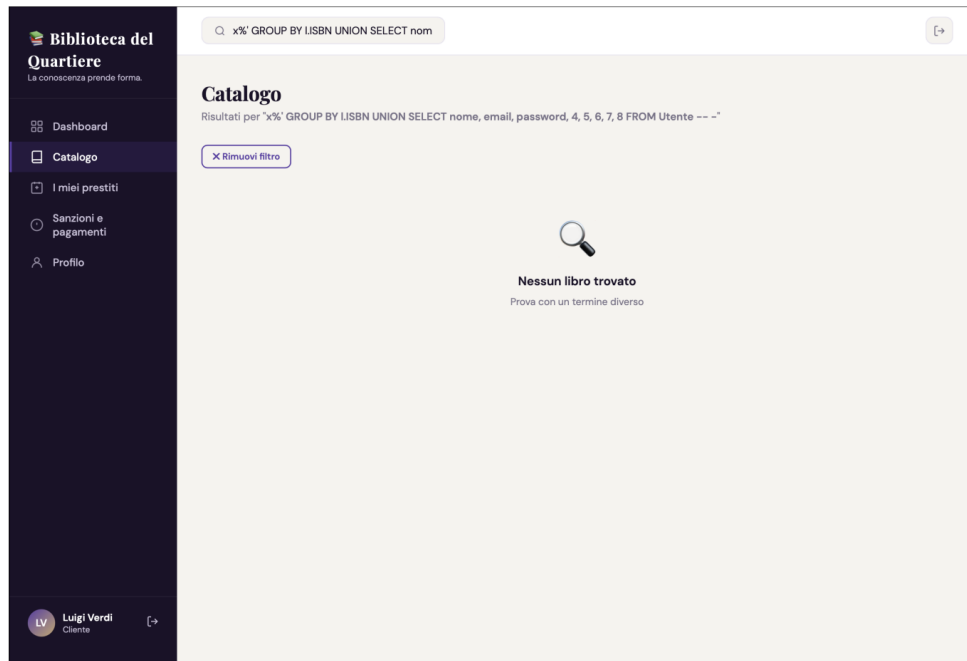


Figura 6: Fallimento della Data Exfiltration: la query parametrizzata non rileva alcun libro corrispondente al payload e restituisce 0 risultati.

4.4 Validazione di Integrità e Disponibilità: Attacco 3 (Piggy-backing)

Il test finale ha riprodotto l'attacco distruttivo basato sull'iniezione di istruzioni multiple (*Stacked Queries*) nel campo *data_scadenza* del modulo prestiti, alterato eludendo la validazione HTML lato client tramite manipolazione del DOM. Il payload iniettato è stato: ...; DELETE FROM Sanzione; -- a. L'efficacia della contromisura poggia su due pilastri tecnici:

1. **Rimozione dell'esecuzione multipla:** L'assenza del parametro `multi=True` nel file `app.py` inibisce l'interprete Python dall'inoltrare query a cascata, neutralizzando il punto e virgola come separatore di istruzioni.
2. **Type Validation a livello DB:** Il driver tenta di mappare l'intero payload testuale all'interno di una colonna configurata rigidamente con il tipo `DATE`.

Il mismatch di tipo genera un'eccezione nativa (`DataError 1292`). L'applicazione interrompe immediatamente la transazione e, tramite l'istruzione `db.rollback()`, annulla qualsiasi modifica parziale garantendo l'Atomicità (ACID) del database:

```
except Exception as ex:
    db.rollback()
    print("Errore inserimento prestito:", ex)
```

Contemporaneamente, il log di backend registra dettagliatamente il tentativo di intrusione. L'output generato dal container Docker documenta l'avvenuta intercettazione e sanitizzazione della minaccia:

```
web-1 | Errore inserimento prestito: 1292 (22007): Incorrect date
value: '2026-12-31', 'in corso'); DELETE FROM Sanzione; -- a'
for column 'data_scadenza' at row 1
```

Mentre nel backend l'errore viene loggato a uso amministrativo, l'attaccante visualizza esclusivamente un reindirizzamento di sicurezza, senza compromettere i dati sottostanti. L'interfaccia grafica conferma quindi che la distruzione strutturale è stata sventata.

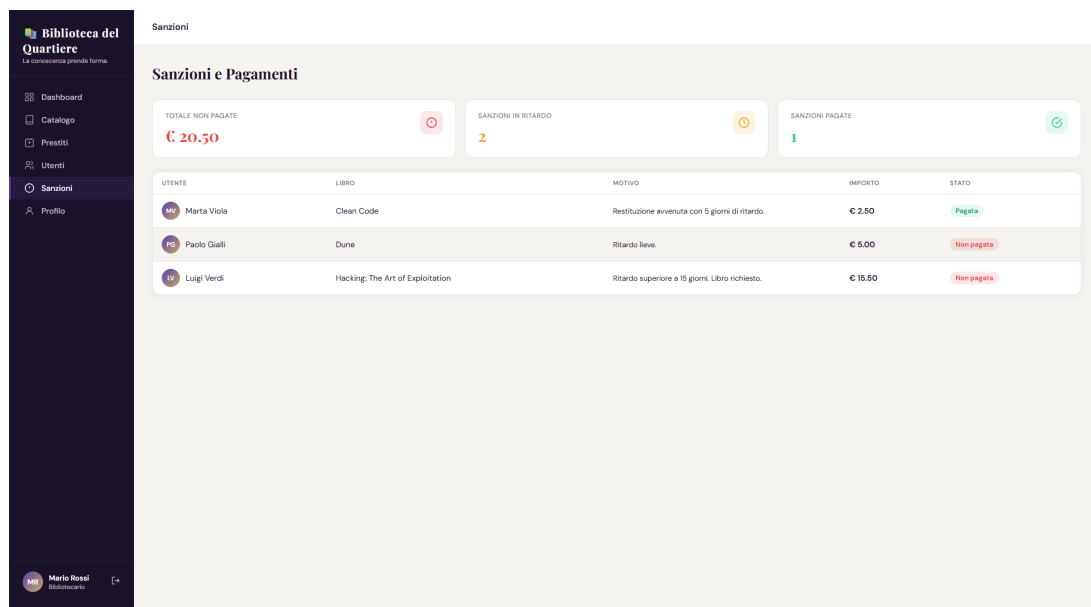


Figura 7: Integrità garantita: l'interfaccia utente conferma che i record delle sanzioni risultano intatti e inalterati a seguito del fallimento dell'attacco Piggybacking.

4.5 Conclusioni

Il Penetration Test finale ha certificato il totale ripristino della sicurezza dell'app "Libreria del Quartiere". La transizione da un approccio basato sulla concatenazione di stringhe all'utilizzo metodico dei *Prepared Statements* ha mitigato alla radice le vulnerabilità della categoria Injection (OWASP Top 10). Inoltre, l'applicazione del Principio del Minimo Privilegio (PoLP) a livello di architettura DBMS, ottenuta de-privilegiando l'utente `web_biblioteca` (rimuovendo i permessi DROP e DELETE sulle strutture critiche), ha fornito un robusto strato di Difesa in Profondità (*Defense in Depth*). Il sistema attuale è in grado di respingere tentativi di *Authentication Bypass*, esfiltrazione dati e distruzione strutturale (*Denial of Service*), operando in un perimetro sicuro che garantisce piena aderenza ai

requisiti della Triade CIA: Confidenzialità delle identità, Integrità dei registri operativi e Disponibilità continua del servizio.